DOCUMENT 01 OF 07 • docs/PRD.md

Product Requirements Document (PRD)

> **MYSchool ERP — JSS School**
> *A clear, high-concurrency plan before the first line of code.*

---

01. Product Overview

Property	Specification Details
Product Name	**MYSchool ERP** (JSS School Edition)
Platform	Responsive Web Application (Next.js 19 App Router, Tailwind CSS, shadcn/ui)
Target Institution	**JSS School** (Central Board, multi-campus physical branches, academic wings)
Core Architecture	Unified Full-Stack Repository, NextAuth.js (Auth.js v5) with Edge JWT cookies, URL-driven branch/wing tenancy (`/[branchSlug]/[deptCode]/[module]`), Managed Supabase PostgreSQL with Supavisor Connection Pooling, Inngest Async Workers
Primary Goal	Digitize, automate, and connect all daily academic and administrative operations across all JSS School campuses into a single unified platform—absorbing peak 8:30 AM attendance spikes, safeguarding financial audits with strict manual UTR verification, enforcing statutory student exit barriers, and guaranteeing lifetime student transcript access.

---

02. Problem & Goal

🎯 DOWNSIDE OF NOT HAVING THIS SYSTEM	🏆 WHAT SUCCESS LOOKS LIKE (DIGITIZES
• 8:30 AM Attendance Bottleneck: Manual paper roll-calls delay legal duty-of-care absence alerts to parents.	• 100ms Bulk Attendance: 40-student si using Prisma `createMany` committed
• Unverified Ledger Drift: Premature or unverified fee deposits cause accounting errors and audit chaos.	• Immutable Financial Audit: Zero auto 2-step UTR clearance by campus bursa
• 504 Gateway Timeouts: Synchronous report card and admit card batch PDF generation hangs HTTP threads.	• Zero-Timeout Batch Engine: Inngest a hundreds of PDFs into Supabase Stora
• Accidental Student Purges: Missing transfer or conduct records during withdrawal leads to regulatory liability	• Statutory Exit Barrier: Hard cryptog requiring verified TC, Character & N
• Bloated Database Backups: Binary files inside SQL.	• Direct Pre-Signed Uploads: 60s clien

---

03. Target User Personas

Persona	Role Scope	Key Responsibilities & Capabilities	Actions / Badges
Super Admin	Central JSS School Board Executive	Group-wide financial auditing, multi-campus consolidated roll calls, cross-branch policy configuration.	`Consolidated View` `Trust Audit` `Global Scoping`
Branch Admin	Campus Principal / Director	Campus-wide admissions, academic cycle initialization, staff duty approvals, exam scheduling.	`Campus Locked` `Staff Approvals` `Cycle Control`
Department Head (HOD)	Academic Wing Leader (e.g. PUC, Primary)	Curriculum pacing, class-to-teacher mapping, period substitutions, leave escalations.	`Wing Locked` `Substitutions` `Leave Signoff`
Class Teacher	Classroom Instructor	8:30 AM morning attendance matrix submission, daily student leave approval, student performance reviews.	`Roster Grid` `Attendance Submit` `Leave Review`
Management	Campus Operations, Bursar & Records	Physical bank statement reconciliation, UTR queue clearance, admission & exit certificate verification, fee ledger auditing.	`UTR Queue` `Ledger Audit` `Doc Clearance` `Exit Barrier`
Parent / Guardian	Household Pupil Guardian	Multi-ward toggling, fee payment submission (UTR + screenshot), leave application, receipt downloads.	`Ward Switcher` `Fee Submit` `Leave Apply`
Student	Enrolled Pupil	Class timetable inspection, exam schedule view, permanent lifetime access to historical report cards.	`Self Service` `Lifetime Grades` `Schedule`

---

04. Core Features Matrix

Concise technical feature listing without unnecessary narrative bloat:

1. **URL-Scoped Tenancy (`/[branchSlug]/[departmentCode]/[module]`):**
   - Independent deep-linking across browser tabs without session or cookie collisions.
   - Layout-level data scoping and isolated Next.js cache boundaries.
   - Global sticky navbar combobox for instantaneous branch/wing switching by cleared personnel.
2. **High-Speed 8:30 AM Attendance Matrix:**
   - Single-payload JSON dispatch containing all 40 student records per classroom.
   - Atomic database write via Prisma `createMany` (or bulk upsert) in a single round-trip.
   - Non-blocking asynchronous trigger firing SMS/push notifications to guardians for unexcused absences.
3. **Manual UTR Fee Verification Pipeline:**
   - Student fee balances remain strictly untouched upon parent transaction initiation.
   - Duplicate UTR check validates that the submitted transaction reference number has never been used.
   - Partial payments fully supported: amount declared is credited against total dues only upon approval.
   - Dedicated Bursar audit desk matches physical bank statements before ledger write and receipt issuance.
4. **Headless Asynchronous Batch PDF Engine (Buzzer Token):**
   - Offloaded rendering of 500–2,000 watermarked admit cards / report cards via Inngest workers.
   - Fast token handoff: server returns `jobId` in ~30ms, preventing browser 504 Gateway Timeouts.

- Background compilation with `@react-pdf/renderer` directly streamed to private Supabase Storage.
- Pollable job log with 24-hour pre-signed ZIP/PDF download ticket upon completion.

5. **Direct Pre-Signed Document Storage & Split-Screen Audit Loop:**
   - Direct-to-bucket upload via 60-second HMAC pre-signed URLs, protecting Next.js server RAM.
   - Split-screen verification interface: management previews documents alongside student data.
   - Mandatory rejection reason: rejecting a document requires a written reason that immediately alerts the parent with a direct re-upload prompt.
6. **Statutory Exit Document Barrier:**
   - Hard programmatic blocker preventing permanent student deletion or archiving.
   - Two-stage confirmation modal requiring staff to type the pupil's exact `admission_no`.
   - Requires 4 verified statutory certificates: Transfer Certificate (TC), Character Certificate, Migration Certificate, and Campus No-Dues Slip.
7. **Single-Parent Multi-Ward Hub & Roster-Linked Leaves:**
   - Single-parent profile model (no dual-parenting overhead) mapped to multiple wards across campuses with top-bar switching.
   - Medical/general leave requests with attachments routed to assigned Class Teacher.
   - Approved student leaves automatically pre-populate class registers as `EXCUSED_LEAVE`.
8. **Dynamic Teacher Allocation & Substitution Roster:**
   - Cross-branch faculty teaching assignments across multiple physical campuses.
   - Automated timetable collision checks when assigning daily substitution duties during teacher absences.
   - Staff attendance tracking logged separately with daily timestamps.
9. **Academic Cycle Lifecycle & Permanent Lifetime Transcripts:**
   - Institution-configured academic calendar cycle start and end dates.
   - Annual promotion workflows batching students to the next grade at cycle close.
   - Past attendance archived to read-only cold storage; students retain permanent lifetime access to download official transcripts and report cards.

---

05. Scope Definition

Must Have (MVP)

- Full-stack Next.js App Router with URL-driven multi-branch routing.
- Supabase PostgreSQL 16+ integration with Supavisor transaction connection pooling.
- Central NextAuth.js RBAC authentication (7 distinct user roles) under single origin.
- High-speed attendance register with single-payload bulk commit (~10ms).
- Manual UTR verification queue with duplicate detection and two-step fee balance deduction.
- Student document upload via pre-signed URLs with audit feedback loop.
- Mandatory exit certificate verification guard prior to student deletion.
- Multi-ward parent switcher and roster-synchronized student leave management.
- Teacher substitution duty assignment for unassigned periods.
- Inngest headless async worker setup for batch PDF compilation.

Later (Post-MVP)

- Biometric hardware fingerprint/RFID integration for automated staff attendance sync.
- Push notification integration with WhatsApp Business API for fee and absence alerts.
- Bus GPS tracking integration for real-time guardian transit visibility.
- Native mobile applications (React Native / Expo) wrapping the existing Next.js backend API contracts.
- AI-assisted fee default forecasting based on historical payment cadences.

---

06. Acceptance Criteria

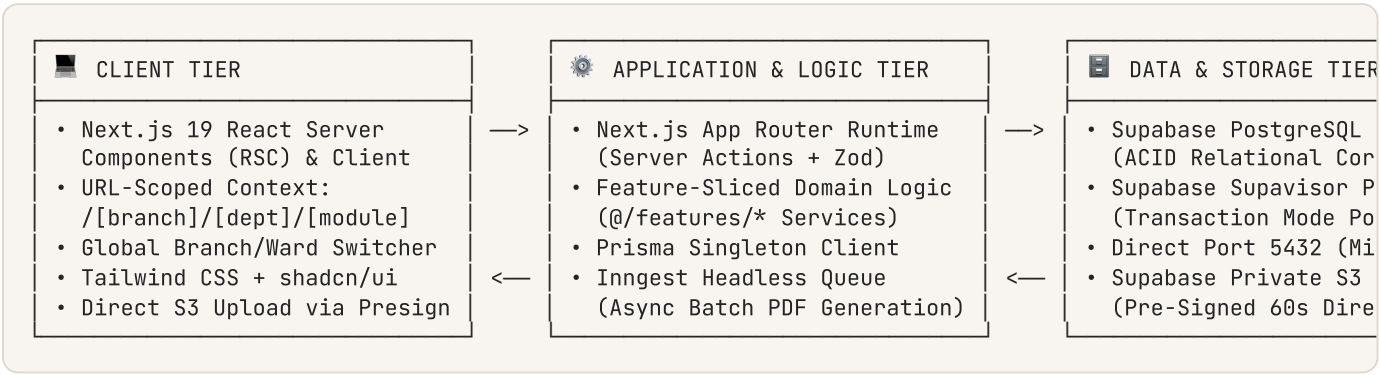
TECHNICAL INTEGRITY	ZERO-TOLERANCE EDGE CASES	BUILD & QUALITY GATES
• URL params define branch/wing; no cross-branch query leaks. • Max 25 physical connections used during 500-user spike via Supavisor transaction mode. • All files uploaded via 60s signed direct-to-S3 URLs. • Prisma singleton used strictly.	• Zero balance deduction on payment submit without UTR sign • Student delete hard-fails if TC or No-Dues slip is missing. • Roster submit dispatches 1 HTTP request per room, not 40. • Inngest job handles 500 PDFs without browser 504 gateway err	• Zero TypeScript errors o `tsc --noEmit`. • Zero ESLint errors or un Prisma client instantiatio • All environment variable validated through Zod sche • Additive-only database migrations applied via Por

Architecture & Data Specification

MYSchool ERP — JSS School

Define how every component, connection pool, and data model connects.

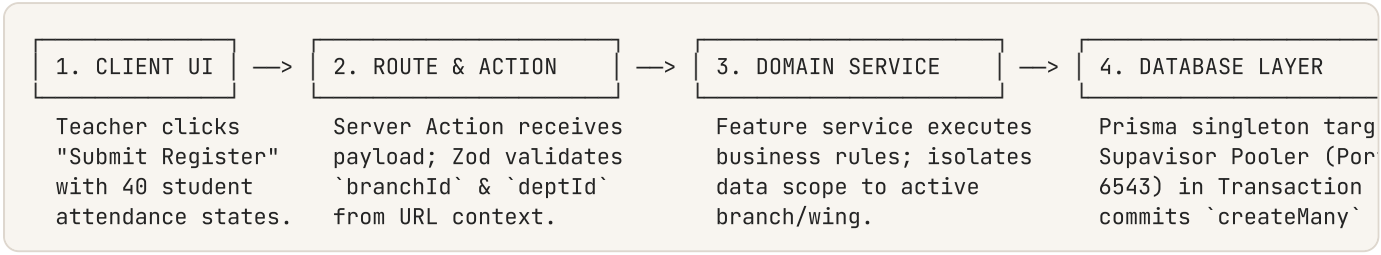
01. System Overview



02. Technology Stack

Layer	Technology	Version / Configuration	Technical Justification
Framework	Next.js (App Router)	React 19 / Next.js 15+	Unified repository eliminates CORS; shares Zod & Prisma types; halves hosting overhead.
Language	TypeScript	Strict Mode (`5.x+`)	End-to-end type safety from database models to frontend form components.
Database	Supabase PostgreSQL	PostgreSQL 16+	Managed ACID compliance, 7-day Point-In-Time Recovery (PITR), zero Linux VM maintenance.
Connection Pooling	Supabase Supavisor	Port `6543` (Transaction Mode)	Multiplexes 500+ concurrent requests into 15–25 physical connections during 8:30 AM peak.
ORM / Query Engine	Prisma ORM	Latest Stable	Type-safe migrations, auto-generated TypeScript models, dual-URL pooler compatibility.
Authentication	NextAuth.js (Auth.js)	v5 (Beta / Latest)	Credentials Provider + Edge JWT session cookies; zero DB hits in middleware.
Async Task Runner	Inngest	Headless Task Queue	Offloads multi-minute PDF rendering from main HTTP threads, preventing 504 timeouts.
Document Compiler	`@react-pdf/renderer`	React-based PDF Engine	Headless generation of watermarked, multi-page admit cards and report card bundles.
Object Storage	Supabase Storage	S3-Compatible Private Buckets	Stores student certificates and receipts outside SQL; pre-signed direct upload bypasses RAM.
Validation Layer	Zod	Runtime Schema Validation	Validates all server actions, form submissions, and environment variables (`@/config/env.ts`).
Styling & Icons	Tailwind CSS + Lucide	Vanilla CSS Variables	Clean, accessible design system matching institutional typography and high-density tables.

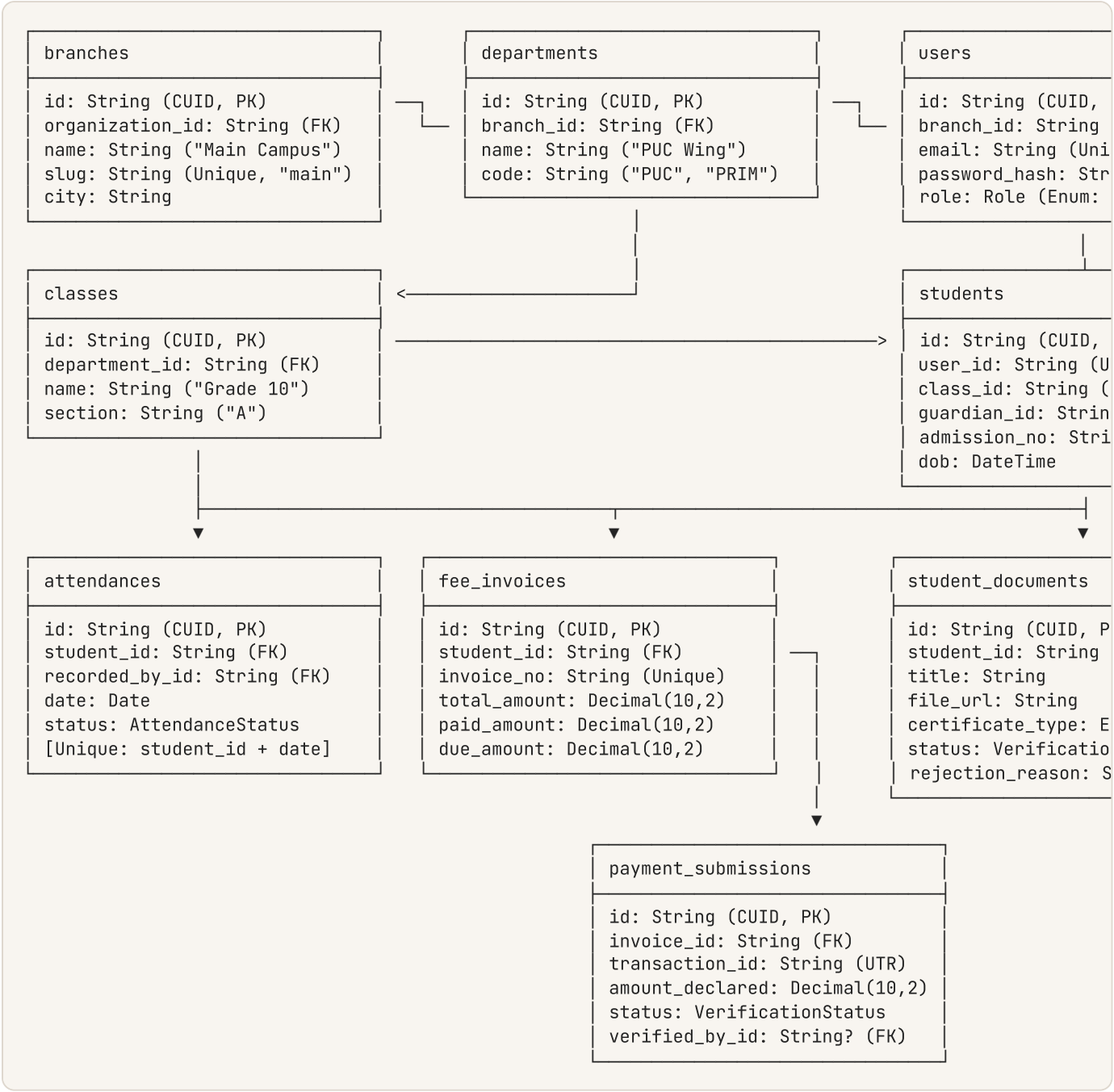
03. Request Lifecycle



Key Request Path Workflows:

1. **Attendance Write:** Client sends 1 array payload $\rightarrow$ Zod validation $\rightarrow$ Service wraps in `db.attendance.createMany({ skipDuplicates: true })` $\rightarrow$ Supavisor executes in 1 socket (~10ms) $\rightarrow$ Return success $\rightarrow$ Trigger Inngest absence notice dispatch.
2. **Fee Verification & Partial Payments:** Parent selects payment gateway/UPI, enters UTR reference + receipt screenshot $\rightarrow$ Duplicate UTR check ensures transaction ID is globally unique $\rightarrow$ Recorded as `PENDING_VERIFICATION` (student balance strictly untouched) $\rightarrow$ Bursar audits physical bank statement $\rightarrow$ Manual `[APPROVE]` atomically decrements `due_amount` by `amount_declared` (supports partial payments) and issues an immutable digital receipt; `[REJECT]` requires a mandatory reason and alerts parent for re-submission.
3. **Document Audit & Rejection Loop:** Direct pre-signed S3 upload from browser $\rightarrow$ Document placed in Management Verification Queue $\rightarrow$ Bursar reviews split-screen document preview $\rightarrow$ `[VERIFY]` marks document approved; `[REJECT]` requires mandatory reason and renders active re-upload prompt on student/parent portal.
4. **Permanent Deletion Barrier:** Delete request $\rightarrow$ Staff must type student's exact `admission_no` into confirmation modal $\rightarrow$ Server Action verifies existence of 4 statutory exit certificates (TC, Character, Migration, No-Dues) marked `VERIFIED` $\rightarrow$ Hard block (HTTP 403) if any are missing or unverified $\rightarrow$ Archival to cold storage upon verified exit.
5. **Batch PDF Render (Buzzer Token Pattern):** Admin requests 2,000 admit cards/hall tickets $\rightarrow$ Next.js hands back `jobId` in ~30ms $\rightarrow$ Inngest headless worker compiles PDFs in chunks using `@react-pdf/renderer` $\rightarrow$ Streams zip/PDF directly to private Supabase Storage $\rightarrow$ Pre-signed 24h download URL saved to `bulk_job_logs` with active progress bar.

04. Relational Data Model Schema



Core Schema Entity Summary:

- `branches`: Physical campuses (Main, North, Tech) with unique routing slugs.
- `departments`: Academic divisions (Primary, Secondary, PUC, Technical) bound to a branch.
- `users`: Consolidated authentication and role assignment (`SUPER_ADMIN`, `BRANCH_ADMIN`, `DEPT_HEAD`, `TEACHER`, `MANAGEMENT`, `STUDENT`, `PARENT`).
- `students`: Enrolled pupils mapped to a user account, class section, and guardian.
- `attendances`: Daily attendance records enforced unique by `[student_id, date]`.
- `fee_invoices` & `payment_submissions`: Two-phase fee ledger ensuring manual UTR reconciliation before balance updates.
- `student_documents`: Academic records, medical certificates, and mandatory exit barriers.
- `substitution_duties`: Dynamic daily teacher period substitutions without altering master timetables.
- `academic_histories`: Immutable lifetime archive for annual report cards and GPAs.
- `bulk_job_logs`: Asynchronous tracking table for headless Inngest batch PDF jobs.

05. Architectural Dependency Rules

```
┌──────────────────────────────────┐ ┌──────────────────────────────────┐ ┌──────────────────────────────────┐
│ 1. DUMB UI, SMART SERVICES       │ │ 2. BUSINESS RULE ENFORCEMENT     │ │ 3. ADDITIVE SCHEMA EVOLUTI       │
├──────────────────────────────────┤ ├──────────────────────────────────┤ ├──────────────────────────────────┤
│ • React components only handle  │ │ • No raw balance decrements.     │ │ • Never run `prisma db pus       │
│   state, events, and rendering.  │ │ • Mandatory exit documents guard │   with force-reset in produc     │
│ • No Prisma queries or SQL       │ │   student deletion.              │ │ • All schema updates must        │
│   execution inside UI components.│ │ • Bulk single-payload attendance │   additive (new optional col     │
│ • All operations route through   │ │   writes strictly enforced.      │   or standalone tables).         │
│   `@/features/*/server/*.service`│ │ • Batch PDFs offloaded to queue. │ │ • Migrations run via Port        │
└──────────────────────────────────┘ └──────────────────────────────────┘ └──────────────────────────────────┘
```

5.1 Standard API Response Envelope

All Route Handlers and Server Actions return a uniform JSON envelope to guarantee predictable client-side state handling and error parsing:

```
export interface ApiResponse<T = any> {
  success: boolean;
  data?: T;
  error?: string;
  metadata?: Record<string, any>;
}
```

5.2 Universal Error Handling & Boundaries

- **Custom Error Hierarchy:** Domain services throw typed exceptions (`AppError` , `UnauthorizedError` , `NotFoundError` , `ValidationError`).
- **Safe Envelope Formatting:** Global try/catch wrappers serialize errors into `{ success: false, error: err.message }` with appropriate HTTP status codes (400, 401, 403, 404, 500).
- **React Error Boundaries:** Feature layouts wrap views with error boundary fallbacks to prevent whole-page crashes if an isolated sub-component fails.

5.3 Feature-Sliced Directory Hierarchy

Code is organized by business feature domains rather than technical file layers:

```
src/
├── app/                          # Next.js App Router (pages, layouts, route handlers)
│   └── [branchSlug]/[deptCode]/  # URL-driven multi-branch context boundaries
├── features/                     # Isolated domain slices
│   ├── attendance/               # components/, server/*.service.ts, schemas/*.zod.ts
│   ├── fees/                     # UTR verification queue, ledger services, receipt layouts
│   ├── documents/                # Split-screen viewer, S3 pre-signed upload handlers
│   ├── students/                 # SIS profile, exit barrier guard, lifetime grade viewer
│   └── substitution/             # Timetable collision checks, daily period substitution
├── components/ui/                # Headless, accessible UI primitives (shadcn/ui)
├── config/                       # Type-safe environment validation via Zod (env.ts)
└── lib/                          # db.ts (Prisma singleton), inngest.ts, storage.ts
```

5.4 Clean Engineering Conventions

- **Naming Standards:** PascalCase for React components/types (`AttendanceRoster.tsx`), camelCase for utilities/services (`verifyUtrPayment.ts`), snake_case for PostgreSQL database entities (`student_documents`).
- **Dumb UI, Smart Services:** React components only manage presentation and local UI state; business logic, database queries, and third-party APIs belong exclusively in `*.service.ts` domain services.

- **Dead-Code Elimination:** Proactive removal of unused imports, dead functions, and commented-out code blocks during every feature cycle.

---

06. Security, Authentication & Route Access Matrix

6.1 Authentication Mechanism: NextAuth.js (Auth.js v5)

- **Strategy:** Credentials Provider with secure, HTTP-only, SameSite=Lax JWT session cookies.
- **Zero Database Bottleneck:** Decodes `userId`, `role`, and `branchId` directly from the signed JWT in Next.js Edge middleware with **0ms database query latency**.
- **Unified Origin:** Shared cookies eliminate cross-origin third-party cookie blocking.

6.2 Route Access & Permission Matrix

Route Pattern	Cleared Roles	Behavioral Guard & Unauthorized Handling
`/[branch]/[dept]/attendance`	`TEACHER`, `DEPT_HEAD`, `BRANCH_ADMIN`	Scoped to assigned classroom; non-teachers redirect to `/unauthorized`.
`/[branch]/[dept]/fees`	`MANAGEMENT`, `BRANCH_ADMIN`, `SUPER_ADMIN`	Bursar UTR audit queue & ledger clearance; 403 Forbidden for teachers/parents.
`/[branch]/[dept]/documents`	`MANAGEMENT`, `BRANCH_ADMIN`	Admission audit & statutory exit barrier verification; 403 Forbidden for others.
`/[branch]/[dept]/leaves`	`TEACHER`, `DEPT_HEAD`, `PARENT`	Scoped strictly to parent's active child or teacher's assigned classroom.
`/[branch]/[dept]/classes`	`BRANCH_ADMIN`, `DEPT_HEAD`	Section roster management & dynamic faculty period substitution duty.
`/admin/consolidated`	`SUPER_ADMIN` exclusively	Group-wide audit & cross-campus reports; 404 Not Found for campus-locked users.

6.3 Environment Configuration Contract (`.env.example`)

- `DATABASE_URL`: `postgresql://postgres:[PWD]@aws-0-[REGION].pooler.supabase.com:6543/postgres?pgbouncer=true` (Port 6543)
- `DIRECT_URL`: `postgresql://postgres:[PWD]@db.[REF].supabase.co:5432/postgres` (Port 5432)
- `AUTH_SECRET`: 32-byte cryptographic secret for NextAuth JWT encryption
- `NEXT_PUBLIC_SUPABASE_URL` & `SUPABASE_SERVICE_ROLE_KEY`: Supabase S3 private storage access
- `INNGEST_EVENT_KEY` & `INNGEST_SIGNING_KEY`: Headless background queue synchronization
- `SMS_GATEWAY_API_KEY`: SMS provider for morning unexcused absence alerts

6.4 Local Development Seed Strategy (`prisma/seed.ts`)

Run `npm run db:seed` on a fresh database to automatically provision the JSS School Central Board organization, 3 campus branches (`main`, `north`, `east`), core academic wings, and 1 demo account per role with pre-hashed credentials.

---

07. Failure Handling & Resilience Matrix

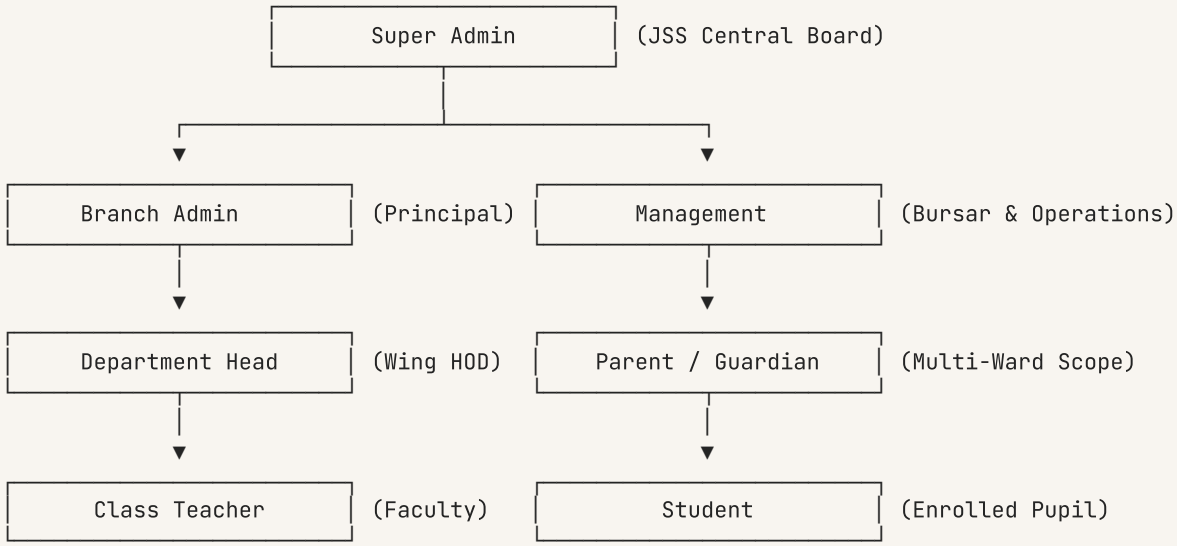
Failure Scenario	Root Cause & Real-World Risk	Architectural Handling Strategy & Safeguard
8:30 AM Attendance Surge	**Serverless Socket Exhaustion:** 100+ teachers submitting at the bell creates hundreds of concurrent serverless threads, exhausting PostgreSQL's ~100 connection limit and throwing `FATAL: remaining connection slots are reserved`.	**Bulk Array Payload + Supavisor Pooling:** Classroom grid dispatches a single array of 40 students committed in ~10ms via Prisma `createMany` (100 total queries vs. 4,000). Supavisor (Port 6543) transaction mode holds physical database sockets only during query execution, multiplexing 500+ teachers across **15–25 physical connections**.
Batch PDF Generation	**HTTP 504 Gateway Timeout:** Compiling 500+ admit cards with student photos, barcodes, and watermarks requires 1–3 minutes of heavy CPU rendering, exceeding the 60s reverse proxy / browser timeout limit.	**Inngest Async Token Pattern:** Next.js offloads rendering to Inngest and returns an instant 200 response with `jobId` in ~30ms. Headless worker compiles PDFs in chunks, streams bundles directly to private Supabase Storage, and issues a 24-hour pre-signed download link to the admin dashboard.
Invalid or Unmatched Fee UTR	**Ledger Drift & Fraudulent Clearance:** Deducting dues prematurely upon parent payment intent causes massive financial discrepancies if the bank transfer fails or the UTR reference is invalid.	**Two-Step Manual Audit Queue:** Submissions are tagged `PENDING_VERIFICATION`; **student balance remains strictly untouched**. The campus bursar verifies the UTR against the school's physical bank statement. Only manual `[VERIFY & APPROVE]` triggers the atomic ledger deduction and generates an immutable receipt.
Unverified Student Deletion	**Regulatory Liability & Lost Transcripts:** Accidental or premature deletion before statutory clearance violates education board statutory mandates and permanently destroys student academic history.	**Statutory Exit Barrier Guard:** Programmatic barrier guards the deletion API. Checks `student_documents` to verify that **Transfer Certificate (TC)**, **Character Certificate**, and **No-Dues Clearance** exist with status `VERIFIED`. Deletion is hard-blocked if any slip is missing.
Local Dev Hot Reload Socket Leak	**Exhausted Connection Slots:** Next.js hot module reloads create new Prisma client instances on every file save during development, rapidly saturating local PostgreSQL ports.	**Global Prisma Singleton:** `src/lib/db.ts` binds PrismaClient to `globalThis`, guaranteeing only one singleton instance is shared across module reloads.

Security & Access Control (SECURITY.md)

> **MYSchool ERP — JSS School**
> *Role-based clearances, identity matrix, cryptographic tokens, and institutional data isolation.*

---

01. Complete Identity Hierarchy (All Roles)



Role Scope Definitions:

1. `SUPER_ADMIN` **(Central Board):** Unrestricted multi-campus visibility. Accesses `/all/all/analytics` for consolidated trust audits, group-wide fee policies, and enrollment analytics.
2. `BRANCH_ADMIN` **(Campus Principal):** Scoped strictly to physical campus (`User.branchId`). Oversees campus admissions, staff allocations, academic cycle promotions, and post-11:00 AM attendance overrides.
3. `DEPT_HEAD` **(Academic Wing HOD):** Scoped to specific academic wing (`branchId` + `departmentId`). Assigns daily period substitutions, reviews student leave escalations, and approves terminal grade books.
4. `TEACHER` **(Classroom Instructor):** Scoped to assigned classroom & section. Submits 8:15–11:00 AM morning attendance matrix, reviews class medical leaves, and enters exam marks.
5. `MANAGEMENT` **(Campus Bursar & Records Officer):** Dedicated financial and administrative clearance. Operates UTR verification queue, executes fee ledger adjustments, issues digital receipts, and audits admission & exit certificates.
6. `PARENT` **(Pupil Guardian):** Scoped to family wards across campuses. Persistent top navbar dropdown toggles between enrolled children; submits fee UTRs and medical leave requests.
7. `STUDENT` **(Enrolled Pupil):** Scoped to personal profile. Inspects timetables, exam dates, fee dues, and enjoys permanent lifetime access to download past grade transcripts.

---

02. Comprehensive Permissions Matrix

Functional Capability	Super Admin	Principal (Branch Admin)	Dept Head (HOD)	Class Teacher	Management (Bursar)	Parent / Student
Cross-Branch Toggling	Unrestricted	Campus Locked	Campus Locked	Campus Locked	Campus Locked	Ward Locked
Class Attendance Submit	View / Audit	View / Override	View / Override	Submit Assigned	None	View Own
Attendance Lock Override	Full Override	Permitted (>11 AM)	With Audit Note	None (Frozen)	None	None
Fee Structure Config	Global Policy	Campus Level	None	None	Create / Modify	None
UTR Verification & Ledger	Full Audit	View Only	None	None	Approve / Reject	Submit UTR
Student Leave Approvals	Full Override	Campus Level	Department Level	Assigned Class	None	Submit Leave
Batch Admit Card Printing	Permitted	Permitted	Permitted	None	None	None
Permanent Student Deletion	Sign-off Only	Execute (Exit Cert)	Blocked	Blocked	Blocked	Blocked
Lifetime Grade Downloads	Full Trust	Campus Level	Department Level	Class Level	None	View / Download

---

03. Route-Based Clearance & Parameter Scoping

Every database query MUST be strictly scoped by the active route segments:

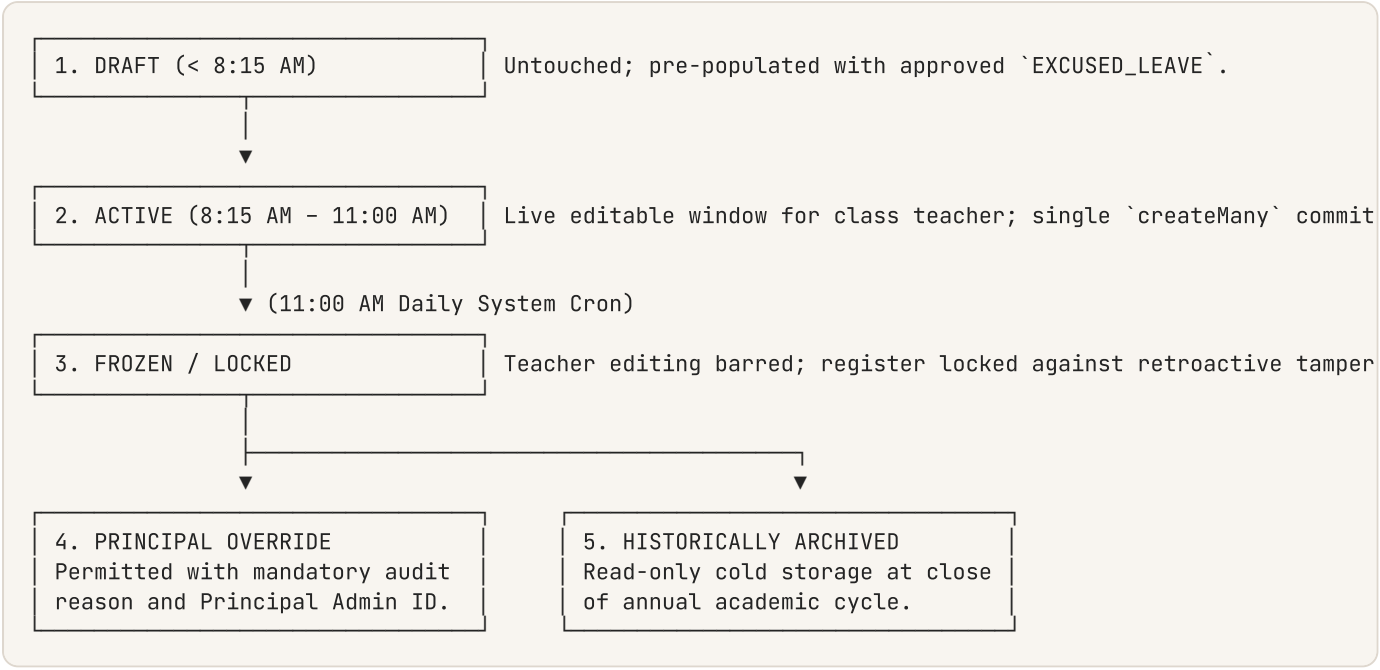
```
GET /[branchSlug]/[departmentCode]/[module]
```

Context Engine Rules:

- **Middleware Verification:** Resolves `branchSlug` $\rightarrow$ `branchId` and `departmentCode` $\rightarrow$ `departmentId`. Non-super-admins attempting to access unauthorized branches receive an immediate `403 Forbidden`.
- **Sub-Path Preservation:** When an authorized administrator switches from `Main Campus → Secondary` to `North Campus → Secondary`, the active leaf route (e.g. `/attendance`) is preserved, navigating directly to `/north/sec/attendance`.
- **Consolidated Trust View:** `/all/all/analytics` is cryptographically restricted to `SUPER_ADMIN` JWT claims, bypassing campus boundaries for group-wide rollups.

---

04. Attendance Time-Lock State Machine



---

05. Statutory Exit Document Barrier & Purge Protection

Hard programmatic blocker on student deletion. Deletion API halts if any required certificate is missing or unverified:

1. **Transfer Certificate (TC):** Valid serial registration number and formal school seal.
2. **Character / Conduct Certificate:** Conduct assessment signed by Department Head.
3. **Migration Certificate:** State / Central education board eligibility slip.
4. **No-Dues Clearance:** Confirmation of ₹0 balance from Management and library clearance.

- **Purge Execution:** Requires typing the student's full Admission ID (`ADM-JSS-2026-XXXX`), selecting a mandatory departure category, and compiling an immutable cold-storage ZIP archive in Supabase Private Storage prior to cascade deletion.

---

06. Pre-Signed Storage Tokens

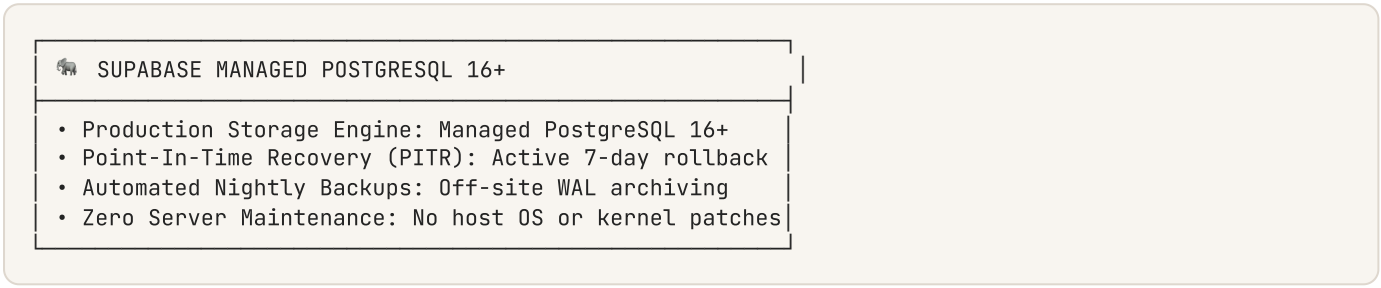
- **Direct Uploads:** Client requests a 60-second HMAC pre-signed S3 upload URL. Next.js server RAM is protected from large binary file streams.
- **Confidential Downloads:** Student report cards, birth certificates, and fee receipts are stored in private buckets and delivered via 24-hour signed download tickets.

Database & Connection Pooling (DATABASE.md)

> **MYSchool ERP — JSS School**
> *PostgreSQL 16+, Supavisor Transaction Multiplexing & High-Concurrency Patterns.*

---

01. Engine & Hosting Topology



---

02. Connection Architecture (The Dual-URL Pattern)

To survive serverless scale and prevent connection crashes during morning peak hours:

Environment Variable	Port	Mode	Intended Consumer	Max Conn Pool
`DATABASE_URL`	`6543`	**Transaction Mode**	Next.js Server Actions & Runtime API	25 physical slots (handles 500+ callers)
`DIRECT_URL`	`5432`	**Session Mode**	Prisma CLI migrations (`prisma migrate`)	Direct connection (requires schema lock)

---

03. High-Concurrency Roll Call (Prisma `createMany`)

```
// src/features/attendance/server/attendance.service.ts
import { db } from "@/lib/db";
import { AttendanceStatus } from "@prisma/client";

interface StudentRosterEntry {
  studentId: string;
  status: AttendanceStatus;
}

export async function submitClassroomAttendance(
  recordedById: string,
  date: Date,
  roster: StudentRosterEntry[]
) {
  // Single-payload commit multiplexed in ~10 milliseconds
  const payload = roster.map((entry) => ({
    studentId: entry.studentId,
    recordedById,
    date,
    status: entry.status,
  }));

  return await db.attendance.createMany({
    data: payload,
    skipDuplicates: true,
  });
}
```

---

04. Additive Migration Directives

1. **Strictly Additive Changes:** Never rename, drop, or mutate existing database columns in production.
2. **Nullable First:** All newly introduced columns must be defined as optional (`?`) or provide a `@default(...)`.
3. **Execution Guard:** Autonomous agents are forbidden from executing `prisma db push --force-reset`.

Design System

> **MYSchool ERP — JSS School**
> *A consistent, warm editorial visual language for institutional clarity.*

---

01. Design Direction

A clean, modern, and timeless aesthetic that feels calm, human, and intentional.

🌿 MINIMAL	🧡 WARM	📐 CONSISTENT
Focused and clutter-free layouts optimized for high density data tables.	Inviting and human-centered tones built on ivory, beige, and wine.	A unified visual experience across all campuses and modules.

---

02. Color Palette

A refined and versatile palette, designed for clarity, warmth, and enduring institutional appeal.

Token Name	Hex Code	Swatch & Usage
White	`#FFFFFF`	Base card surfaces, modal sheets, and high-contrast containers.
Ivory	`#FAF7F2`	Primary page canvas background and subtle contrast layers.
Beige / Sand	`#E7DCCB`	Secondary panel backgrounds, subtle dividers, and hover fills.
Burgundy	`#682038`	Primary brand accent, numbered badges, primary buttons, and key callouts.
Charcoal	`#242424`	Primary high-contrast typography, icons, and strong border strokes.
Success Olive	`#2E7D32`	Verified statuses, approved leaves, paid receipts, and pass marks.
Warning Red	`#BA1A1A`	Rejection alerts, missing documents, unexcused absences, and delete barriers.

---

03. Typography

`Inter` is a clean, modern typeface that balances readability with a friendly, institutional feel.

Level	Size	Weight	Line Height	Usage
Heading XL	`40px`	Bold (`700`)	`48px`	Page hero titles and document headers
Heading Large	`28px`	Semibold (`600`)	`36px`	Section titles (`01` , `02` headings)
Subheading	`20px`	Medium (`500`)	`28px`	Card headers and modal titles
Body Regular	`16px`	Regular (`400`)	`24px`	Standard prose, form descriptions
Small / Label	`14px`	Regular / Medium	`20px`	Table headers, badges, metadata
Mono (Code)	`12px`	Medium (`500`)	`16px`	UTR codes, admission numbers, IDs (`JetBrains Mono`)

---

04. Spacing Scale

A consistent 4px/8px modular spacing scale creates rhythmic harmony and clear visual hierarchy.

Spacing Token	Pixels	CSS Variable	Common Usage
`space-1`	`4px`	`0.25rem`	Badge padding, icon offsets
`space-2`	`8px`	`0.5rem`	Form control padding, gap between pills
`space-3`	`12px`	`0.75rem`	Card internal cell spacing
`space-4`	`16px`	`1.0rem`	Default container padding, table cell padding
`space-6`	`24px`	`1.5rem`	Section gutters, grid gaps
`space-8`	`32px`	`2.0rem`	Major module separation, page margins

---

05. Border Radius

Consistent corner radii create a soft, modern, and approachable silhouette.

Token	Radius	Applied Elements
`radius-sm`	`4px`	Monospaced tags, small input badges, table selection checks
`radius-md`	`8px`	Standard action buttons, text input fields, dropdown menus
`radius-lg`	`12px`	Content cards, modal sheets, lifecycle node boxes
`radius-xl`	`24px`	Large hero banners, preview containers
`radius-full`	`9999px`	Numbered circular badges (`01` , `02`), pill status chips

---

06. Core UI Components

Buttons

- **Primary:** Burgundy background (`#682038`), White text, 8px radius, subtle scale-down on active.
- **Secondary:** Transparent background, 1px Beige border (`#E7DCCB`), Charcoal text (`#242424`).
- **Destructive:** Warning Red (`#BA1A1A`), White text, requires multi-step confirmation.

Status Chips & Badges

- **Verified / Present:** Light Olive surface (`#E8F5E9`), Dark Green text (`#2E7D32`), 9999px radius.
- **Pending Verification:** Light Amber surface (`#FFF8E1`), Deep Amber text (`#F57F17`).
- **Rejected / Absent:** Light Red surface (`#FFEBEE`), Dark Red text (`#BA1A1A`).

Input Fields & Controls

- **Background:** Pure White (`#FFFFFF`) with 1px Beige stroke (`#E7DCCB`).
- **Focus State:** 2px Burgundy outline (`#682038`) with zero layout shift.

---

07. Visual States

- **Empty States:** Gentle line-art illustration, clear descriptive subtitle, single call-to-action button.
- **Loading States:** Shimmer skeleton cards adhering to the exact grid dimensions to eliminate layout shifts.
- **Error States:** Border changes to Warning Red (`#BA1A1A`) with contextual inline error messages below the field.

---

08. Responsive Grid Breakpoints

- **Mobile (`375px+`):** Single column card stacks; sticky bottom action bars for attendance submit.
- **Tablet (`768px+`):** 2-column modular grid; collapsible left sidebar navigation.
- **Desktop (`1024px+`):** Fixed 280px navigation rail, fluid 12-column content area, maximum reading width `1440px` .

---

09. Accessibility Standards

- **Readable Contrast:** All text pairings strictly achieve WCAG AA contrast ratio ($\ge 4.5:1$).
- **Keyboard Navigation:** Full tab-index traversal with visible burgundy focus rings (`:focus-visible`).
- **Clear Labels & ARIA:** Every icon button includes an explicit `aria-label` and screen-reader helper tag.

AI Agent Instructions (AGENTS.md)

> **MYSchool ERP — JSS School**
> *Give your coding agent a clear, disciplined, and deterministic way to work.*

01. Project Context

```
Read the docs. Understand the code. Implement with intent.
This project is an institutional multi-campus ERP for JSS School.
All architectural decisions, database models, and operational constraints are fixed.
Never guess or introduce generic patterns that contradict this dossier.
```

02. Source of Truth

Document	File Path	Canonical Responsibility
PRD.md	`docs/PRD.md`	Feature scopes, user personas, MVP boundary, acceptance criteria.
ARCHITECTURE.md	`docs/ARCHITECTURE.md`	Tech stack, request lifecycle, data flow, connection pool rules, failure modes.
schema.prisma	`prisma/schema.prisma`	Strict relational database models, enums, indexes, and foreign keys.
DESIGN_SYSTEM.md	`docs/DESIGN_SYSTEM.md`	Palette, typography, spacing tokens, component hierarchy, accessibility rules.

03. Implementation Workflow



04. Working Rules

✅ DO (MANDATORY PRACTICES)	❌ AVOID (STRICT ANTI-PATTERNS)
• Scrutinize branch and department route parameters: Every query MUST include `branchId` & `departmentId`. • Use the Prisma singleton from `@/lib/db.ts`: Runtime queries use Supavisor (Port 6543). • Execute single-payload bulk writes: Use `db.attendance.createMany` for classroom rosters. • Route batch PDF tasks to Inngest background workers: Keep Next.js HTTP responses under 50ms. • Enforce statutory exit barrier: Verify TC, Character, and No-Dues before deletion. • Validate all environment variables with Zod schemas: Import from `@/config/env.ts`.	• NEVER run `prisma db push --force-re columns. Migrations must be strictly • NEVER instantiate `new PrismaClient(code. It bypasses pooling and causes • NEVER auto-deduct fee dues upon paym Only staff approval can clear a stud • NEVER render batch PDFs synchronousl threads. This causes 504 Gateway Tim • NEVER delete a student record if req certificates are missing or unverifi • NEVER access `process.env` directly • NEVER write business logic or Prisma

05. Core Engineering & Hygiene Directives

Adhere strictly to these 10 baseline architectural and coding standards:

1. **Feature-Sliced Architecture:** Organize code strictly under `src/features/[feature]/` (`components/`, `server/*.service.ts`, `schemas/*.zod.ts`).
2. **Dumb UI, Smart Services:** UI components only handle render logic and local interaction state. All business logic, Prisma calls, and mutations belong in `*.service.ts`.
3. **Dead-Code Elimination:** Proactively remove unused imports, dead functions, test debris, and commented-out code blocks during every pass.
4. **Strict Naming Conventions:**
   - PascalCase for React components and types (`StudentRosterTable.tsx`).
   - camelCase for functions, Server Actions, hooks, and utilities (`submitAttendanceAction.ts`).
   - snake_case for PostgreSQL database tables and columns (`student_documents`, `invoice_no`).
5. **Universal Error Handling:** Use custom error classes (`AppError`, `UnauthorizedError`, `NotFoundError`, `ValidationError`) and wrap domain operations in try/catch envelopes returning uniform error responses.
6. **Environment Separation via Zod:** Never read `process.env` directly in feature files. Import strictly from `@/config/env.ts` with runtime Zod parsing.
7. **Modular UI Primitives:** Leverage accessible, composable primitives from `@/components/ui/` (shadcn/ui + Tailwind CSS).
8. **Deterministic State Management:** Rely on URL route parameters (`/[branchSlug]/[deptCode]/...`) as primary source of truth for branch context, supplemented by React Server Components and lightweight Zustand for client UI state.
9. **Inline Code Documentation:** Annotate complex concurrency constraints, transaction boundaries, and permission assertions with concise JSDoc comments.
10. **Scalable API Envelopes:** Enforce uniform JSON envelopes (`{ success, data, error, metadata }`) across all Route Handlers and Server Actions.

06. Validation Checklist

Before marking any task, feature, or PR complete, verify every gate:

- [ ] **Typecheck Passes:** `npm run typecheck` (`tsc --noEmit`) outputs **zero** errors.
- [ ] **Lint Clean:** `npm run lint` passes without warnings or unused imports.

- [ ] **Supavisor Pool Integrity:** No discrete `PrismaClient` instances created outside `@/lib/db.ts`.
- [ ] **URL Scope Enforced:** All data queries are bound to the active branch slug and department code.
- [ ] **Financial Audit Protected:** Fee submission leaves `due_amount` unchanged; status set to `PENDING_VERIFICATION`.
- [ ] **Duplicate UTR Blocked:** Payment submission rejects duplicate transaction reference numbers.
- [ ] **Roster Bulk Written:** Attendance registers commit in a single database payload using `createMany`.
- [ ] **Exit Documents Verified:** Permanent deletion actions check for TC, Character, Migration, and No-Dues records.

---

07. Conflict Handling Protocol

```
┌──────────────────────────────────────────────────────────────────────┐
│ ⚠️ CONFLICT RESOLUTION ESCALATION                                    │
├──────────────────────────────────────────────────────────────────────┤
│ If an implementation requirement is ambiguous or conflicts with code:  │
│                                                                      │
│ 1. Review Canonical Docs: Re-read `PRD.md` and `ARCHITECTURE.md`.    │
│ 2. Inspect Prisma Schema: Check `prisma/schema.prisma` for type rules. │
│ 3. Formulate Viable Options: Identify 2 concrete options with impacts. │
│ 4. Halt and Request Human Sign-off: Present the conflict directly to  │
│    the team rather than guessing or applying destructive workarounds.  │
└──────────────────────────────────────────────────────────────────────┘
```

---

08. Definition of Done (DoD)

```
┌──────────────────────────┐ ┌──────────────────────────┐ ┌──────────────────────────┐ ┌──────────
│ 1. REQUIREMENTS MET      │ │ 2. CODE IS TYPE-SAFE     │ │ 3. ZERO REGRESSIONS      │ │ 4. DOCUMEN
├──────────────────────────┤ ├──────────────────────────┤ ├──────────────────────────┤ ├──────────
│ The feature satisfies all │ │ Zero TypeScript errors;  │ │ Existing branch context  │ │ Any update
│ explicit criteria from   │ │ Zod schemas validate both │ │ and database isolation   │ │ schema cha
│ the PRD and dossier.     │ │ inputs and outputs.      │ │ remain intact.           │ │ in docs an
└──────────────────────────┘ └──────────────────────────┘ └──────────────────────────┘ └──────────
```

DOCUMENT 07 OF 07 • prisma/schema.prisma

Authoritative Relational Database Schema (prisma/schema.prisma)

> **MYSchool ERP — JSS School**
> *PostgreSQL 16+ Relational Schema with Multi-Tenancy & Financial Precision*

```
datasource db {
  provider  = "postgresql"
  url       = env("DATABASE_URL")  // Supavisor Transaction Pooler (Port 6543)
  directUrl = env("DIRECT_URL")    // Direct Session Connection (Port 5432)
}

generator client {
  provider = "prisma-client-js"
}

// ==================== ENUMS ====================

enum Role {
  SUPER_ADMIN      // JSS School Central Board Executive
  BRANCH_ADMIN     // Campus Principal / Director
  DEPT_HEAD        // Head of Department / Wing Head (HOD)
  TEACHER          // Classroom Instructor
  MANAGEMENT       // Campus Management / Bursar & Operations
  STUDENT          // Enrolled Pupil
  PARENT           // Guardian
}

enum AttendanceStatus {
  PRESENT
  ABSENT
  LATE
  EXCUSED_LEAVE
}

enum VerificationStatus {
  PENDING_VERIFICATION
  VERIFIED
  REJECTED
}

enum CertificateType {
  TRANSFER_CERTIFICATE
  CHARACTER_CERTIFICATE
  MIGRATION_CERTIFICATE
  MEDICAL_CLEARANCE
  NO_DUES_SLIP
}

enum LeaveStatus {
  PENDING
  APPROVED
  REJECTED
}

// ==================== INSTITUTIONAL HIERARCHY ====================

model Organization {
  id        String   @id @default(cuid())
  name      String   // "JSS School"
  createdAt DateTime @default(now()) @map("created_at")
  updatedAt DateTime @updatedAt @map("updated_at")

  branches  Branch[]

  @@map("organizations")
}

model Branch {
  id             String       @id @default(cuid())
  organizationId String       @map("organization_id")
  name           String       // "Main Campus", "North Campus", "Tech Campus"
  slug           String       @unique // "main", "north", "tech"
  city           String
```

```
  createdAt      DateTime     @default(now()) @map("created_at")

  organization   Organization @relation(fields: [organizationId], references: [id], onDelete: Cascade)
  departments    Department[]
  users          User[]
  academicCycles AcademicCycle[]
  staffAttendance StaffAttendance[]

  @@index([organizationId])
  @@map("branches")
}

model Department {
  id        String   @id @default(cuid())
  branchId  String   @map("branch_id")
  name      String   // "Primary Wing", "Secondary Wing", "PUC"
  code      String   // "PRIM", "SEC", "PUC", "TECH"
  createdAt DateTime @default(now()) @map("created_at")

  branch    Branch   @relation(fields: [branchId], references: [id], onDelete: Cascade)
  classes   Class[]

  @@unique([branchId, code])
  @@index([branchId])
  @@map("departments")
}

// ==================== USER ACCOUNTS & RBAC ====================

model User {
  id               String            @id @default(cuid())
  branchId         String            @map("branch_id")
  email            String            @unique
  passwordHash     String            @map("password_hash")
  firstName        String            @map("first_name")
  lastName         String            @map("last_name")
  phone            String?
  role             Role              @default(STUDENT)
  createdAt        DateTime          @default(now()) @map("created_at")

  branch           Branch            @relation(fields: [branchId], references: [id], onDelete: Cascade
  studentProfile   Student?          @relation("StudentUser")
  parentWards      Student[]         @relation("ParentWards")
  verifiedPayments PaymentSubmission[] @relation("PaymentAuditor")
  verifiedDocs     StudentDocument[] @relation("DocumentAuditor")
  recordedAttendance Attendance[]    @relation("AttendanceTaker")
  reviewedLeaves   LeaveApplication[] @relation("LeaveApprover")
  substituteDuties SubstitutionDuty[] @relation("SubstituteTeacher")
  absentDuties     SubstitutionDuty[] @relation("AbsentTeacher")
  staffAttendance  StaffAttendance[]

  @@index([branchId])
  @@map("users")
}

// ==================== ACADEMICS & ROSTERS ====================

model AcademicCycle {
  id        String    @id @default(cuid())
  branchId  String    @map("branch_id")
  name      String    // "Academic Year 2026-2027"
  startDate DateTime  @map("start_date")
  endDate   DateTime  @map("end_date")
  isCurrent Boolean   @default(false) @map("is_current")

  branch    Branch    @relation(fields: [branchId], references: [id], onDelete: Cascade)
  history   AcademicHistory[]

  @@unique([branchId, name])
```

```
  @@map("academic_cycles")
}

model Class {
  id           String     @id @default(cuid())
  departmentId String     @map("department_id")
  name         String     // "Grade 10"
  section      String     // "A"
  createdAt    DateTime   @default(now()) @map("created_at")

  department   Department @relation(fields: [departmentId], references: [id], onDelete: Cascade)
  students     Student[]
  substitutions SubstitutionDuty[]

  @@unique([departmentId, name, section])
  @@index([departmentId])
  @@map("classes")
}

model Student {
  id             String            @id @default(cuid())
  userId         String            @unique @map("user_id")
  classId        String            @map("class_id")
  guardianId     String            @map("guardian_id") // Maps to parent User record
  admissionNo    String            @unique @map("admission_no")
  dob            DateTime
  createdAt      DateTime          @default(now()) @map("created_at")

  user           User              @relation("StudentUser", fields: [userId], references: [id], onDele
  guardian       User              @relation("ParentWards", fields: [guardianId], references: [id], on
  class          Class             @relation(fields: [classId], references: [id])
  documents      StudentDocument[]
  attendances    Attendance[]
  invoices       FeeInvoice[]
  leaves         LeaveApplication[]
  history        AcademicHistory[]

  @@index([classId])
  @@index([guardianId])
  @@map("students")
}

// ==================== WORKFLOW ENTITIES ====================

model Attendance {
  id           String           @id @default(cuid())
  studentId    String           @map("student_id")
  recordedById String           @map("recorded_by_id")
  date         DateTime         @db.Date
  status       AttendanceStatus @default(PRESENT)
  createdAt    DateTime         @default(now()) @map("created_at")

  student      Student          @relation(fields: [studentId], references: [id], onDelete: Cascade)
  recordedBy   User             @relation("AttendanceTaker", fields: [recordedById], references: [id])

  @@unique([studentId, date])
  @@index([studentId, date])
  @@index([date])
  @@map("attendances")
}

model StaffAttendance {
  id           String           @id @default(cuid())
  userId       String           @map("user_id")
  branchId     String           @map("branch_id")
  date         DateTime         @db.Date
  status       AttendanceStatus @default(PRESENT)
  checkIn      DateTime?        @map("check_in")
  checkOut     DateTime?        @map("check_out")
```

```
  user         User             @relation(fields: [userId], references: [id], onDelete: Cascade)
  branch       Branch           @relation(fields: [branchId], references: [id], onDelete: Cascade)

  @@unique([userId, date])
  @@index([branchId, date])
  @@map("staff_attendances")
}

model LeaveApplication {
  id           String      @id @default(cuid())
  studentId    String      @map("student_id")
  startDate    DateTime    @db.Date @map("start_date")
  endDate      DateTime    @db.Date @map("end_date")
  reason       String
  proofUrl     String?     @map("proof_url")
  status       LeaveStatus @default(PENDING)
  reviewedById String?     @map("reviewed_by_id")
  reviewNotes  String?     @map("review_notes")

  student      Student     @relation(fields: [studentId], references: [id], onDelete: Cascade)
  reviewedBy   User?       @relation("LeaveApprover", fields: [reviewedById], references: [id])

  @@index([studentId])
  @@map("leave_applications")
}

model FeeInvoice {
  id          String        @id @default(cuid())
  studentId   String        @map("student_id")
  invoiceNo   String        @unique @map("invoice_no")
  totalAmount Decimal       @db.Decimal(10, 2) @map("total_amount")
  paidAmount  Decimal       @default(0.00) @db.Decimal(10, 2) @map("paid_amount")
  dueAmount   Decimal       @db.Decimal(10, 2) @map("due_amount")
  dueDate     DateTime      @map("due_date")
  createdAt   DateTime      @default(now()) @map("created_at")

  student     Student       @relation(fields: [studentId], references: [id], onDelete: Cascade)
  payments    PaymentSubmission[]

  @@index([studentId])
  @@map("fee_invoices")
}

model PaymentSubmission {
  id              String             @id @default(cuid())
  invoiceId       String             @map("invoice_id")
  transactionId   String             @map("transaction_id") // Bank Ref / UTR
  amountDeclared  Decimal            @db.Decimal(10, 2) @map("amount_declared")
  receiptFileUrl  String?            @map("receipt_file_url")
  status          VerificationStatus @default(PENDING_VERIFICATION)
  rejectionReason String?            @map("rejection_reason")
  verifiedById    String?            @map("verified_by_id")
  verifiedAt      DateTime?          @map("verified_at")
  createdAt       DateTime           @default(now()) @map("created_at")

  invoice         FeeInvoice         @relation(fields: [invoiceId], references: [id], onDelete: Cascad
  verifiedBy      User?              @relation("PaymentAuditor", fields: [verifiedById], references: [

  @@index([invoiceId])
  @@index([transactionId])
  @@map("payment_submissions")
}

model StudentDocument {
  id              String             @id @default(cuid())
  studentId       String             @map("student_id")
  title           String             // "Birth Certificate"
  fileUrl         String             @map("file_url")
```

```
  certificateType CertificateType?   @map("certificate_type") // For exit barriers
  status          VerificationStatus @default(PENDING_VERIFICATION)
  rejectionReason String?            @map("rejection_reason")
  verifiedById    String?            @map("verified_by_id")
  verifiedAt      DateTime?          @map("verified_at")
  createdAt       DateTime           @default(now()) @map("created_at")

  student         Student            @relation(fields: [studentId], references: [id], onDelete: Cascad
  verifiedBy      User?              @relation("DocumentAuditor", fields: [verifiedById], references:

  @@index([studentId])
  @@map("student_documents")
}

model SubstitutionDuty {
  id                  String   @id @default(cuid())
  date                DateTime @db.Date
  periodNumber        Int      @map("period_number")
  classId             String   @map("class_id")
  absentTeacherId     String   @map("absent_teacher_id")
  substituteTeacherId String   @map("substitute_teacher_id")

  class               Class    @relation(fields: [classId], references: [id])
  absentTeacher       User     @relation("AbsentTeacher", fields: [absentTeacherId], references: [id])
  substituteTeacher   User     @relation("SubstituteTeacher", fields: [substituteTeacherId], reference

  @@unique([date, periodNumber, classId])
  @@map("substitution_duties")
}

model AcademicHistory {
  id               String        @id @default(cuid())
  studentId        String        @map("student_id")
  academicCycleId  String        @map("academic_cycle_id")
  classSnapshot    String        @map("class_snapshot") // e.g., "Grade 9-A"
  finalGpa         String?       @map("final_gpa")
  reportCardPdfUrl String        @map("report_card_pdf_url") // Lifetime student download link
  createdAt        DateTime      @default(now()) @map("created_at")

  student          Student       @relation(fields: [studentId], references: [id], onDelete: Cascade)
  cycle            AcademicCycle @relation(fields: [academicCycleId], references: [id], onDelete: Casc

  @@unique([studentId, academicCycleId])
  @@index([studentId])
  @@map("academic_histories")
}

model BulkJobLog {
  id            String   @id @default(cuid())
  branchId      String   @map("branch_id")
  requestedById String   @map("requested_by_id")
  jobType       String   @map("job_type") // e.g., "ADMIT_CARDS", "FEE_RECEIPTS"
  fileUrl       String   @map("file_url")
  status        String   @default("PENDING") // PENDING, COMPLETED, FAILED
  itemCount     Int      @default(0) @map("item_count")
  createdAt     DateTime @default(now()) @map("created_at")

  @@index([branchId])
  @@map("bulk_job_logs")
}
```